

DESIGN AND IMPLEMENTATION OF ORDERS AND ORDER_DETAILS TABLES

1. Introduction

An Order Management System is an important part of an e-commerce database. It is used to store and manage information about customer orders and the products included in each order.

The **ORDERS** table stores the main information about an order, such as the order ID, customer ID, order date, total amount, and order status. The **ORDER_DETAILS** table stores the individual products purchased in each order, including product ID, quantity, and price.

The two tables are connected using keys. The CUSTOMER_ID connects the order with the customer, while ORDER_ID connects the order with its order details.

2. Objectives

The main objectives are:

- To design the ORDERS table.
- To design the ORDER_DETAILS table.
- To store customer order information.
- To store products associated with each order.
- To maintain relationships between customers, orders, and products.
- To use primary keys and foreign keys for data integrity.
- To retrieve order information using SQL queries and joins.

3. Software and Tools Used

- Database: Oracle Database
- Interface: SQL*Plus command-line client
- Language: Structured Query Language (SQL) with DDL, DML and DQL commands

4. Relationship Between the Tables

One customer can place many orders, and one order can contain many products. This gives a one-to-many relationship between CUSTOMERS and ORDERS, and a one-to-many relationship between ORDERS and ORDER_DETAILS. Each row of ORDER_DETAILS refers to one product in one order, and the PRODUCT_ID column links it to the PRODUCTS table.

Parent Table	Child Table	Linking Column
CUSTOMERS	ORDERS	CUSTOMER_ID
ORDERS	ORDER_DETAILS	ORDER_ID
PRODUCTS	ORDER_DETAILS	PRODUCT_ID

5. Structure of the ORDERS Table

Column	Data Type	Constraint	Description
ORDER_ID	VARCHAR2(10)	PRIMARY KEY	Unique ID of the order
CUSTOMER_ID	VARCHAR2(10)	FOREIGN KEY, NOT NULL	Customer who placed the order
ORDER_DATE	DATE	NOT NULL	Date of the order
TOTAL_AMOUNT	NUMBER(10,2)	CHECK (>= 0)	Total value of the order
ORDER_STATUS	VARCHAR2(20)	DEFAULT 'Pending'	Current status of the order

6. Structure of the ORDER_DETAILS Table

Column	Data Type	Constraint	Description
ORDER_DETAIL_ID	VARCHAR2(10)	PRIMARY KEY	Unique ID of the order line
ORDER_ID	VARCHAR2(10)	FOREIGN KEY, NOT NULL	Order the line belongs to
PRODUCT_ID	VARCHAR2(10)	FOREIGN KEY, NOT NULL	Product that was ordered
QUANTITY	NUMBER(5)	CHECK (> 0)	Units of the product ordered
PRICE	NUMBER(10,2)	NOT NULL	Price of one unit

8. Creating the ORDERS Table

The following DDL statement creates the ORDERS table. It assumes the CUSTOMERS table

```
SQL> CREATE TABLE Orders (  
2     ORDER_ID VARCHAR2(8),  
3     CUSTOMER_ID VARCHAR2(8),  
4     ORDER_DATE DATE,  
5     TOTAL_AMOUNT NUMBER(10,3),  
6     ORDER_STATUS VARCHAR2(25)  
7 );
```

already exists.

```
Table created.
```

9. Creating the ORDER_DETAILS Table

The ORDER_DETAILS table is created next. It assumes the PRODUCTS table already exists.

```
SQL> CREATE TABLE Orders (
2     ORDER_ID VARCHAR2(8),
3     CUSTOMER_ID VARCHAR2(8),
4     ORDER_DATE DATE,
5     TOTAL_AMOUNT NUMBER(10,3),
6     ORDER_STATUS VARCHAR2(25)
7 );
```

Table created.

10. Inserting Records

Sample records are inserted into both tables. The order is inserted first because ORDER_DETAILS depends on it.

Table 1: Records in the ORDERS table

```
SQL> INSERT INTO ORDERS VALUES('0201','C201',TO_DATE('22-08-2026','DD-MM-YYYY'),750,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES('0202','C202',TO_DATE('23-08-2026','DD-MM-YYYY'),1000,'Shipped');
1 row created.

SQL> INSERT INTO ORDERS VALUES('0203','C203',TO_DATE('24-08-2026','DD-MM-YYYY'),500,'Delivered');
1 row created.
```

The stored data is shown below.

Table 2: Records in the ORDER_DETAILS table

```
SQL> INSERT INTO ORDER_DETAILS
2 VALUES ('OD201', '0201', 'P201', 2, 500);

1 row created.

SQL> INSERT INTO ORDER_DETAILS
2 VALUES ('OD202', '0202', 'P202', 3, 600);

1 row created.

SQL> INSERT INTO ORDER_DETAILS
2 VALUES ('OD203', '0203', 'P203', 4, 700);

1 row created.
```

11. Retrieving Data

The records of each table can be displayed using the SELECT statement.

```
SELECT * FROM Orders;
```

```
SQL> SELECT * FROM ORDERS;
```

ORDER_ID	CUSTOMER	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
0201	C201	22-AUG-26	750	Pending
0202	C202	23-AUG-26	1000	Shipped
0203	C203	24-AUG-26	500	Delivered

```
SQL> COMMIT;
```

12. Retrieving Data Using a Join

To display each order together with the products it contains, the two tables are joined on ORDER_ID.

```
SQL> SELECT C.CUSTOMER_ID,  
2         O.ORDER_ID,  
3         O.ORDER_DATE,  
4         O.TOTAL_AMOUNT,  
5         O.ORDER_STATUS AS STATUS  
6 FROM ORDERS O  
7 JOIN CUSTOMER C  
8 ON TO_CHAR(C.CUSTOMER_ID) = REPLACE(O.CUSTOMER_ID, 'C', '');
```

CUSTOMER_ID	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	STATUS
201	0201	22-AUG-26	750	Pending
202	0202	23-AUG-26	1000	Shipped
203	0203	24-AUG-26	500	Delivered

The join returns one row for every order line, showing the customer, product, quantity and price against the order they belong to.

13. Product-wise Sales Summary Using GROUP BY

To find how many times each product was ordered and the total quantity sold, the ORDER_DETAILS table is grouped by PRODUCT_ID. COUNT(*) gives the number of order lines for the product and SUM(Quantity) gives the total units sold.

```
SQL> SELECT Product_ID, COUNT(*) AS No_OF_Times_Ordered, SUM(Quantity) AS Total_Quantity_Sold FROM Order_Details GROUP BY Product_ID;
```

PRODUCT_	NO_OF_TIMES_ORDERED	TOTAL_QUANTITY_SOLD
P202	1	3
P201	1	2
P203	1	4

Figure 1: Output of the product-wise sales summary query

14. Result and Observations

Product_ID	No_OF_Times_Ordered	Total_Quantity_Sold
P202	1	3
P201	1	2
P203	1	4

- Three distinct products, P201, P202 and P203, were ordered.
- Each product was ordered exactly once, so the order count is 1 for all of them.
- P203 recorded the highest quantity sold (4 units), followed by P202 (3 units) and P201 (2 units).
- Because each product appears in only one order line, its total quantity equals the quantity of that single line.

15. Conclusion

The **ORDERS** and **ORDER_DETAILS** tables form an important part of an e-commerce Order Management System. The ORDERS table stores the main information about customer orders, while the ORDER_DETAILS table stores the individual products included in each order.

Primary keys uniquely identify records, while foreign keys establish relationships between customers, orders, and products. SQL commands such as CREATE TABLE, INSERT, SELECT, and JOIN can be used to implement and retrieve the required information.